

CHAPTER 8 — POINTERS AND ARRAYS

SLIDE 1 — WHAT ARE POINTERS AND ARRAYS?

MEET POINTERS AND ARRAYS!

An **array** stores multiple values of the same data type.

A **pointer** can be used to access the elements of an array.

Simple Definition

An array stores multiple values, and a pointer can point to and move through those values.

LOOK AT THIS!

```
None
int a[] = {10, 20, 30, 40};
```

The array contains:

```
None
10    20    30    40
↑
a
```

The name `a` represents the starting location of the array.

We can create a pointer:

None

```
int *p = a;
```

Now:

None

10 20 30 40

↑

p

EASY ANALOGY

Think of an array as a row of boxes .

None



10 20 30 40

↑



The pointer is like a person standing at one box.

It can move from one box to another.

★ REMEMBER!

Array → Stores many values 

Pointer → Moves through the values 

SLIDE 2 — ACCESSING ARRAY ELEMENTS USING A POINTER

HOW CAN WE ACCESS ARRAY VALUES?

Suppose:

None

```
int a[] = {10, 20, 30};  
int *p = a;
```

Since **p** points to the first element:

None

***p**

gives:

None

10

1 2
3 4 **LOOK AT THIS!**

None

```
printf("%d", *p);
```

Output:

None

10

Now move the pointer:

None

p++;

The pointer moves to the next element:

None

```
10      20      30  
      ↑  
      p
```

Now:

None

```
printf("%d", *p);
```

Output:

None

20

EASY ANALOGY

Imagine walking along a row of boxes  :

None

 10 →  20 →  30



*p → Read the current box 

p++ → Move to the next box 

MEMORY TRICK

*p → GET CURRENT VALUE 

p++ → MOVE TO NEXT 

★ TAKEAWAY

A pointer can be used to access array elements one by one.

SLIDE 3 — POINTERS AND ARRAY INDEXES

 CAN WE USE POINTERS WITH ARRAY INDEXES?

Yes! 🎯

Consider:

None

```
int a[] = {10, 20, 30, 40};  
int *p = a;
```

We can access elements using:

None

```
*(p + 0)
```

None

```
10
```

None

```
*(p + 1)
```

None

```
20
```

None

```
*(p + 2)
```

None

```
30
```

None

```
*(p + 3)
```

None

40

LOOK AT THIS!

None

Array:

10	20	30	40
↑	↑	↑	↑
p	p+1	p+2	p+3

So:

None

$*(p + 0) \rightarrow 10$

$*(p + 1) \rightarrow 20$

$*(p + 2) \rightarrow 30$

$*(p + 3) \rightarrow 40$

EASY ANALOGY

Think of p as your starting position  .

None

START

↓

10 → 20 → 30 → 40

$p + 1 \rightarrow$ move one box 

$p + 2 \rightarrow$ move two boxes 

$p + 3 \rightarrow$ move three boxes 

★ REMEMBER!

$*(p + i) \rightarrow$ Access the element at position i .

SLIDE 4 — USING POINTERS TO TRAVERSE AN ARRAY

 CAN WE USE A LOOP?

Yes! 

We can use a pointer to move through the array.

None

```
int a[] = {10, 20, 30, 40};
int *p = a;

for (int i = 0; i < 4; i++)
{
    printf("%d ", *(p + i));
}
```

OUTPUT:

None

```
10 20 30 40
```

 HOW DOES IT WORK?

None

```
i = 0 → *(p + 0) → 10
i = 1 → *(p + 1) → 20
i = 2 → *(p + 2) → 30
i = 3 → *(p + 3) → 40
```

 EASY ANALOGY

Think of the pointer as a **tour guide** .

The array is a row of places:

None

 10 →  20 →  30 →  40

The loop tells the pointer:

Go to position 0 → position 1 → position 2 → position 3.

BIG MEMORY TRICK

None

ARRAY

↓

POINTER

↓

p + i

↓

*(p + i)

↓

GET VALUE 🔍

★ TAKEAWAY

Pointers can be used to traverse and access array elements.

SLIDE 5 — POINTERS AND ARRAYS RECAP

WHAT DID WE LEARN?

1. ARRAY

None

```
int a[] = {10, 20, 30};
```

Stores multiple values.

2. POINTER TO ARRAY

None

```
int *p = a;
```

Points to the first element.

3. ACCESS CURRENT ELEMENT

None

```
*p
```

Gets the current value.

4. MOVE THROUGH ARRAY

None

```
p++;
```

Moves to the next element.

5. ACCESS USING POSITION

None

```
*(p + i)
```

Gets the value at position `i`.

 **ONE BIG MEMORY TRICK!**

None

ARRAY

↓

`int a[] = {10,20,30,40}`

↓

POINTER

↓

`int *p = a`

↓

ACCESS

↓

`*p`

↓

MOVE

↓

`p++`

↓

POSITION

↓

`*(p + i)`

★ REMEMBER!

Array → MANY VALUES 📦

Pointer → ACCESS / MOVE 🔍

`*(p + i)` → VALUE AT POSITION `i`



QUESTIONS

1) What does `p = a` do?

- A) Deletes the array
- B) Makes `p` point to the first element of the array
- C) Moves `p` to the last element

D) Changes all array values

 **Hint:** `a` represents the starting location of the array.

 **Glossary:**

Array → A collection of multiple values stored together.

2 What is the output?

None

```
int a[] = {10, 20, 30};
```

```
int *p = a;
```

```
printf("%d", *p);
```

A) 10

B) 20

C) 30

D) Address of `a`

 **Hint:** `p` initially points to the first element.

 **Glossary:**

First element → The element at position `0` in an array.

3 Which statement accesses the fourth element of the array using a pointer?

A) `*p`

B) `*(p + 1)`

C) `*(p + 2)`

D) `*(p + 3)`

 **Hint:** Array positions start from `0`.

 Glossary:

Index → The position number of an element in an array.

4 Complete the Code

None

```
int a[] = {10, 20, 30, 40};

int *p = a;

printf("%d", _____);
```

Print the **third element** using the pointer.

 **Hint:** Use `*(p+i)`.

 Glossary:

Pointer arithmetic → Using arithmetic operations to move a pointer through an array.

5 What is the output?

None

```
int a[] = {10, 20, 30, 40};

int *p = a;

for (int i = 0; i < 4; i++)
{
    printf("%d ", *(p + i));
}
```

A) 10 20 30 40

B) 0 1 2 3

C) 40 30 20 10

D) 10 30 20 40

💡 **Hint:** $*(p+i)$ accesses each element one by one.

📖 **Glossary:**

Traversal → Visiting each element of an array one by one.